电机的驱动与编码器读取-IIC

1.1 开篇说明

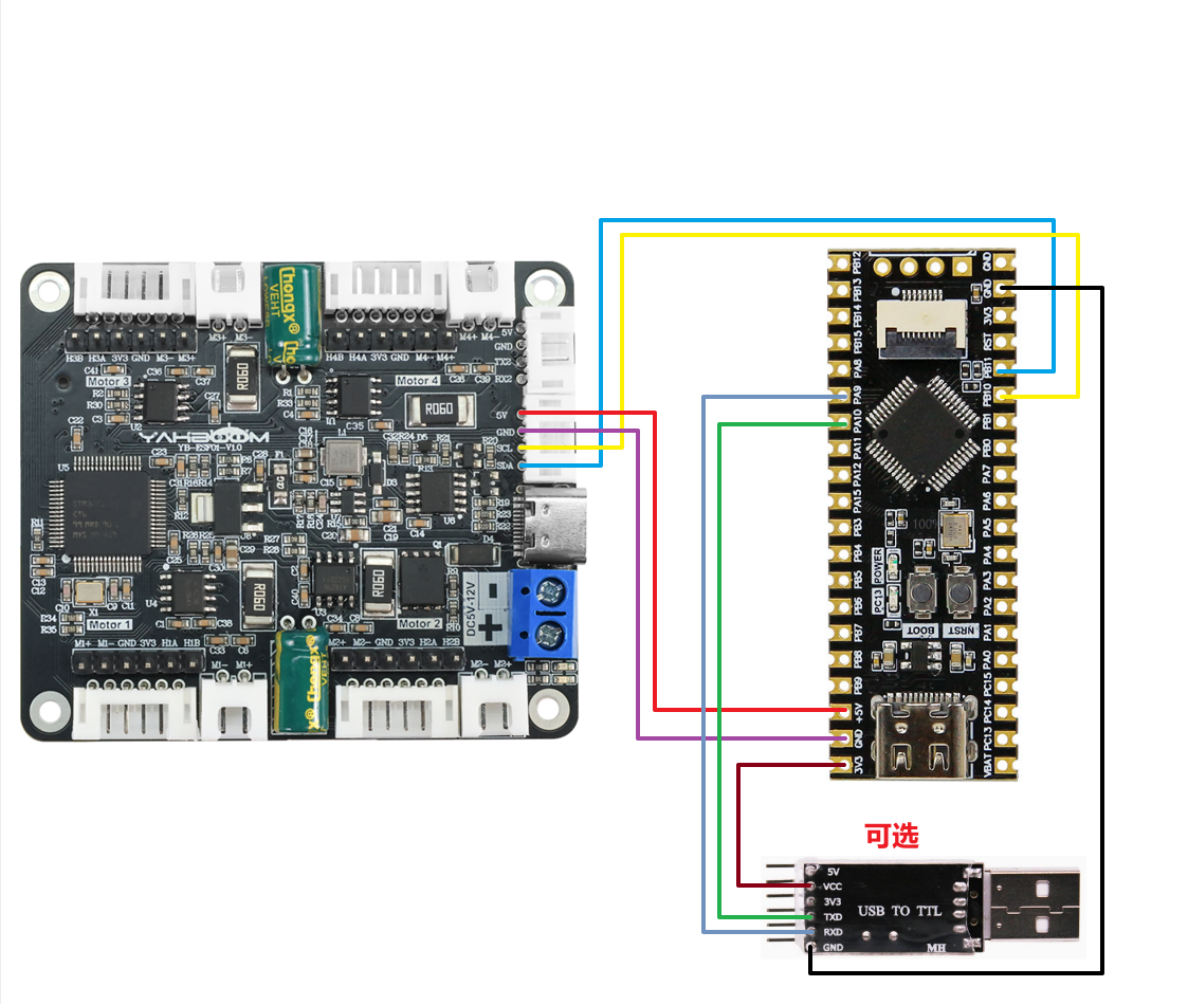
请先阅读《常用电机介绍以及用法》，了解清楚自己现使用的电机参数、接线方式、供电电压。以免造成烧坏主板或者电机的后果。

I2C与串口通讯不能共用，只能选择其中一个。STM32C8T6与RCT6都可兼容使用本工程。

4个电机接口对应小车的关系如下：

M1 -> 左上电机(小车的左前轮)
M2 -> 左下电机(小车的左后轮)
M3 -> 右上电机(小车的右前轮)
M4 -> 右下电机(小车的右后轮)

硬件接线：



四路电机驱动板	STM32C8T6/STM32RCT6
SDA	PB11
SCL	PB10
GND	GND

四路电机驱动板	STM32C8T6/STM32RCT6
5V	5V

电机	四路电机驱动板(Motor)
M2	M-
V	3V3
A	H1B
B	H1A
G	GND
M1	M+

此处接USB转TTL串口模块，是用于打印处理过后的编码器数据的。如果使用的是在本店购买的STM32，可以直接通过TYPE-C接口与电脑USB口相连，也能使用串口打印，就不需要再接USB转TTL串口模块了。

USB TO TTL	STM32C8T6/STM32RCT6
VCC	3V3
GND	GND
RXD	PA9
TXD	PA10

串口配置：**波特率115200、无奇偶校验、无硬件流控、1位停止位**

注：此处的串口用于在串口助手上打印数据，不是用于与驱动板通讯

1.2 代码解释

```
#define UPLOAD_DATA 2  // 1:接收总的编码器数据 2:接收实时的编码器 1: Receive total encoder data 2: Receive real-time encoder data

#define MOTOR_TYPE 1   //1:520电机 2:310电机 3:测速码盘TT电机 4:TT直流减速电机 5:L型520电机
                       //1:520 motor 2:310 motor 3:speed code disc TT motor 4:TT DC reduction motor 5:L type 520 motor
```

- UPLOAD_DATA：用于设置接收电机编码器的数据，设置1为接收编码器总脉冲数，2为10ms的实时脉冲数据。
- MOTOR_TYPE：用于设置使用的电机类型，根据自己现使用的电机对照着注释修改对应的数字即可，不需要再修改代码其余部分。

如果是需要驱动电机并观察数据，只修改程序开头的这两个数字就可以了，代码的其余部分不需要做任何更改。

```
    #if MOTOR_TYPE == 1
    Set_motor_type(1);//配置电机类型  Configure motor type
    delay_ms(100);
    Set_Pluse_Phase(30);//配置减速比 查电机手册得出 Configure the reduction ratio.
Check the motor manual to find out
    delay_ms(100);
    Set_Pluse_line(11);//配置磁环线 查电机手册得出  Configure the magnetic ring wire.
Check the motor manual to get the result.
    delay_ms(100);
    Set_Wheel_dis(67.00);//配置轮子直径,测量得出      Configure the wheel diameter
and measure it
    delay_ms(100);
    Set_motor_deadzone(1900);//配置电机死区,实验得出  Configure the motor dead zone,
and the experiment shows
    delay_ms(100);

    #elif MOTOR_TYPE == 2
    Set_motor_type(2);
    delay_ms(100);
    Set_Pluse_Phase(20);
    delay_ms(100);
    Set_Pluse_line(13);
    delay_ms(100);
    Set_Wheel_dis(48.00);
    delay_ms(100);
    Set_motor_deadzone(1600);
    delay_ms(100);

    #elif MOTOR_TYPE == 3
    Set_motor_type(3);
    delay_ms(100);
    Set_Pluse_Phase(45);
    delay_ms(100);
    Set_Pluse_line(13);
    delay_ms(100);
    Set_Wheel_dis(68.00);
    delay_ms(100);
    Set_motor_deadzone(1250);
    delay_ms(100);

    #elif MOTOR_TYPE == 4
    Set_motor_type(4);
    delay_ms(100);
    Set_Pluse_Phase(48);
    delay_ms(100);
    Set_motor_deadzone(1000);
    delay_ms(100);

    #elif MOTOR_TYPE == 5
    Set_motor_type(1);
    delay_ms(100);
    Set_Pluse_Phase(40);
    delay_ms(100);
    Set_Pluse_line(11);
    delay_ms(100);
```

```
    Set_wheel_dis(67.00);
    delay_ms(100);
    Set_motor_deadzone(1900);
    delay_ms(100);
    #endif
```

这里用于存储本店在售电机的参数，通过修改上方的MOTOR_TYPE参数，就可以实现一键配置。正常情况下使用本店的电机不要修改此处的代码。如果使用的是自己的电机，或者说某项数据需要根据自己的需求来修改，那么可以查看文档《4路电机驱动板控制指令》来了解每一项指令的具体含义。

```
    while(1)
    {
        if(times>=250)//定时器每10ms累加一次，当达到250次，即2.5秒的时候转换一次小车的状态
The timer accumulates every 100ms, and when it reaches 25 times, that is, every
2.5 seconds, the state of the car is changed.
        {
            #if MOTOR_TYPE == 4
            Car_Move_PWM();
            #else
            Car_Move();
            #endif
            times = 0;
        }

        #if UPLOAD_DATA == 1
        Read_ALL_Enconder();
        printf("M1:%d\t M2:%d\t M3:%d\t M4:%d\t
\r\n",Encoder_Now[0],Encoder_Now[1],Encoder_Now[2],Encoder_Now[3]);
        #elif UPLOAD_DATA == 2
        Read_10_Enconder();
        printf("M1:%d\t M2:%d\t M3:%d\t M4:%d\t
\r\n",Encoder_Offset[0],Encoder_Offset[1],Encoder_Offset[2],Encoder_Offset[3]);
        #endif
    }
```

程序中设置了一个10ms的定时器，每一次的定时器中断触发，都会使times这个变量加一，当达到250次，即2.5秒的时候，改变小车的运动状态。如果电机类型选择的是4，即无编码器的电机，那使用pwm版本的小车状态切换函数。同时读取驱动板发送上来的数据，并一边将数据打印出来。

```
//读取相对时间的编码器的数据 10ms的   Read the data of the encoder of relative time
10ms
void Read_10_Enconder(void)
{
    static int8_t buf[2];

    //M1电机编码器的数据    M1 motor encoder data
    i2cRead(Motor_model_ADDR, READ_TEN_M1Enconer_REG, 2, buf);
    Encoder_Offset[0] = buf[0]<<8|buf[1];

    //M2电机编码器的数据    M2 motor encoder data
    i2cRead(Motor_model_ADDR, READ_TEN_M2Enconer_REG, 2, buf);
    Encoder_Offset[1] = buf[0]<<8|buf[1];
```

```
    //M3电机编码器的数据    M3 motor encoder data
    i2cRead(Motor_model_ADDR, READ_TEN_M3Enconer_REG, 2, buf);
    Encoder_Offset[2] = buf[0]<<8|buf[1];

    //M4电机编码器的数据    M4 motor encoder data
    i2cRead(Motor_model_ADDR, READ_TEN_M4Enconer_REG, 2, buf);
    Encoder_Offset[3] = buf[0]<<8|buf[1];

}

//读取电机转动的编码器数据  Read the encoder data of the motor rotation
void Read_ALL_Enconder(void)
{
    static uint8_t buf[2];
    static uint8_t buf2[2];

    //M1电机编码器的数据    M1 motor encoder data
    i2cRead(Motor_model_ADDR, READ_ALLHigh_M1_REG, 2, buf);
    i2cRead(Motor_model_ADDR, READ_ALLLOW_M1_REG, 2, buf2);

    Encoder_Now[0] = buf[0]<<24|buf[1]<<16|buf2[0]<<8|buf2[1];

    //M2电机编码器的数据    M2 motor encoder data
    i2cRead(Motor_model_ADDR, READ_ALLHigh_M2_REG, 2, buf);
    i2cRead(Motor_model_ADDR, READ_ALLLOW_M2_REG, 2, buf2);
    Encoder_Now[1] = buf[0]<<24|buf[1]<<16|buf2[0]<<8|buf2[1];

    //M3电机编码器的数据    M3 motor encoder data
    i2cRead(Motor_model_ADDR, READ_ALLHigh_M3_REG, 2, buf);
    i2cRead(Motor_model_ADDR, READ_ALLLOW_M3_REG, 2, buf2);
    Encoder_Now[2] = buf[0]<<24|buf[1]<<16|buf2[0]<<8|buf2[1];

    //M4电机编码器的数据    M4 motor encoder data
    i2cRead(Motor_model_ADDR, READ_ALLHigh_M4_REG, 2, buf);
    i2cRead(Motor_model_ADDR, READ_ALLLOW_M4_REG, 2, buf2);
    Encoder_Now[3] = buf[0]<<24|buf[1]<<16|buf2[0]<<8|buf2[1];

}
```

从驱动板获取数据后，对它进行移位处理，需要移位才能获取正确数据。

1.3 实验现象

接线正确后，将程序烧录进主板中，复位后可以看见小车会前进2.5秒，后退2.5秒，右旋2.5秒，左旋2.5秒，然后停下2.5秒，又继续以上的状态切换动作。同时在串口助手里能看见打印的电机数值在不断的变化。

M1:1118,M2:1127,M3:1136,M4:1119

M1:1118,M2:1127,M3:1136,M4:1119

M1:1118,M2:1127,M3:1136,M4:1119

M1:1118,M2:1127,M3:1136,M4:1119

M1:1118,M2:1127,M3:1136,M4:1119

M1:1118,M2:1127,M3:1136,M4:1119

M1:1118,M2:1127,M3:1136,M4:1119

M1:1118,M2:1127,M3:1136,M4:1119

M1:1118,M2:1127,M3:1136,M4:1119

M1:1118,M2:1127,M3:1136,M4:1119

M1:1118,M2:1127,M3:1136,M4:1119

M1:1118,M2:1127,M3:1136,M4:1119

M1:1118,M2:1127,M3:1136,M4:1119